Phased GPO-to-Intune Transition Program & Sequencing Roadmap

Implementation Solution Guide

Version 1.0.0 · 2026-06-30

Contents

Executive Summary — Phased GPO-to-Intune Transition Program & Sequencing Roadmap 4
What you get . . . 4
What this delivers . . . 4
How this complements the rest of the catalog . . . 4
The standard deployment model . . . 5
Outcome . . . 5
1. Solution Overview — Phased GPO-to-Intune Transition Program & Sequencing Roadmap 5
What this solution is . . . 5
Why sequencing is the whole point (the *why* lives here) . . . 5
Coexistence & precedence (MDM wins over GP) . . . 6
Conflict and loop guardrails . . . 6
Go/no-go gates per phase . . . 7
The program operating model (roles & KPIs) . . . 7
How this complements the rest of the catalog (no duplication) . . . 8
Honest limitations . . . 8
The standard deployment model . . . 8
2. Requirements Guide — Phased GPO-to-Intune Transition Program 8
2.1 Intune licensing & admin access . . . 8
2.2 Devices enrolled / co-managed in Intune . . . 9
2.3 A rationalized GPO backlog . . . 9
2.4 Group Policy analytics access (for the mechanics phases) . . . 9
2.5 Coexistence precedence decision (MDM wins over GP) . . . 9
2.6 Phase scope / wave groups . . . 10
2.7 Change control, coexistence window & per-phase rollback . . . 10
2.8 Stakeholder alignment on sequencing (security comes later) . . . 10
Checklist . . . 10
3. Implementation Guide — Phased GPO-to-Intune Transition Program 11
The phase sequence (do not reorder) . . . 11
Phase 0 — Rationalize (Assess) . . . 12
Phase 1 — Core OS: set precedence, then migrate the foundation (Configure + Pilot) . . . 12
Phase 2 — Stabilize (Validate + Expand core OS) — THE security go/no-go gate . . . 13
Phase 3 — Security control A: Firewall (gated) . . . 14
Phase 4 — Security control B: Application control (gated) . . . 14
Phase 5 — Expand & decommission GPO scope . . . 15
5. Testing Guide — Phased GPO-to-Intune Transition Program 16

Test 1 — Precedence in effect (MDM wins over GP) . . . 16
Test 2 — Core-OS parity on pilot . . . 16
Test 3 — No conflicts or evaluation loops . . . 16
Test 4 — Stabilization gate (the security go/no-go) . . . 16
Test 5 — Firewall phase: parity + no lockouts . . . 16
Test 6 — App-control phase: parity + no required-app block-outs . . . 16
Test 7 — One authority per setting (decommission readiness) . . . 17
Test 8 — Phase report / KPIs . . . 17
Pass/fail gates . . . 17
6. Troubleshooting Guide — Phased GPO-to-Intune Transition Program 17
A. Settings flip between GPO and Intune (the loop signature) . . . 17
B. Same-setting conflicts on a migrated area . . . 18
C. Pressure to start security before stabilization passes . . . 18
D. Firewall phase: network lockout after migration . . . 18
E. App-control phase: required apps blocked . . . 18
F. A phase won't validate / gate stays red . . . 18
G. Diagnostics quick reference . . . 19
H. When to escalate . . . 19
7. Best Practices — Phased GPO-to-Intune Transition Program 19
8. Deliverables Summary — Phased GPO-to-Intune Transition Program 20
Documentation . . . 20
PowerShell scripts . . . 20
What the buyer receives . . . 21
How this fits the catalog (complements, does not duplicate) . . . 21
Outcome . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary — Phased GPO-to-Intune Transition Program & Sequencing Roadmap

White-label · Migrations

The fastest way to break a Group Policy-to-Intune transition is to migrate security controls — the firewall, application control, attack-surface rules — before the foundation is clean. When core OS settings, precedence, and conflict governance are not yet in place, layering security on top produces policy conflicts, evaluation loops, and lockouts that take a device — or a fleet — offline. Internal pressure to "just move security too, while we're in there" is the single most common cause of a stalled or reversed migration.

This solution is the **program and sequencing roadmap** for doing the transition right. It is the strategy, sequencing, and governance layer that orchestrates the move in a deliberate order: **rationalize the GPO backlog, migrate core OS settings first, stabilize, then layer in security controls** — each behind a go/no-go gate, with coexistence governance so MDM (Intune) wins over Group Policy, and conflict/loop guardrails that prevent two authorities from fighting over the same device.

This is a **deploy-it-as-a-program** package written for a program lead and a small team of admins running a controlled transition across live, domain-joined production devices.

What you get

- A **Solution Overview** of the phased transition model, coexistence and precedence (MDM wins over GP), conflict/loop governance, the go/no-go gate model, and the program operating model (roles, KPIs).
- A **Requirements Guide** with verify / implement-if-missing for every prerequisite — Intune, Entra ID, a rationalized GPO backlog, Group Policy analytics, change control, and stakeholder alignment.
- An **Implementation Guide** — the ordered program runbook: define phases and sequencing, set the coexistence/precedence guardrails, run the core-OS phase, stabilize and validate, sequence the security-control phases behind go/no-go gates, then expand and decommission GPO scope.
- Five **PowerShell scripts** for transition-readiness discovery, per-phase coexistence/conflict validation, a phase-status and conflict report, safe remediation of precedence conflicts, and a phase/wave scope-group builder.
- **Testing** and **Troubleshooting** guides with explicit pass/fail gates, plus field-proven **Best Practices** for the operating model.

What this delivers

A transition that holds together: core OS settings migrated and stabilized first, security controls sequenced behind go/no-go gates, conflicts and loops governed, and one clear authority per setting — so internal pressure to rush does not break the move, and every phase is proven on a pilot before it scales.

How this complements the rest of the catalog

This is the **orchestration layer**. It does not duplicate the per-setting mechanics (covered by the **Group Policy to Intune Settings Migration** solution) or the per-control build-outs (covered by the firewall and application-control migration solutions). It is the program that decides **what order** those happen in, **when** each is safe to start, and **how** the whole transition is governed.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — applied as a phased program, with a go/no-go gate at every phase boundary.

Outcome

A sequenced, governed GPO-to-Intune transition: foundation first, security next, gated throughout — stable, reversible, and evidenced.

1. Solution Overview — Phased GPO-to-Intune Transition Program & Sequencing Roadmap

What this solution is

This is a **program** — a strategy, sequencing, and governance roadmap — for moving an organization off **Group Policy (GPO)** and onto **Microsoft Intune** in a deliberate, gated order. It is not a single wizard and it is not the per-setting migration mechanics. It is the **operating model** that decides *what moves in what order*, *when each phase is safe to start*, and *how the whole transition is governed* so it stays stable while two policy authorities (Group Policy and Intune/MDM) coexist on the same fleet.

The program orchestrates the transition through a fixed sequence of phases:

1. **Rationalize** the GPO backlog — retire dead, duplicate, and obsolete policy before migrating anything, so you migrate intent, not accumulated cruft.
1. **Core OS first** — migrate the foundational, low-blast-radius OS settings (device restrictions, user experience, baseline configuration) and establish the coexistence/precedence guardrails.
1. **Stabilize** — let the fleet settle on the new authority, prove there are no conflicts or evaluation loops, and pass a stabilization gate before touching anything sensitive.
1. **Security controls — later, and in order** — only after the foundation is stable, sequence the higher-risk control families (for example the firewall first, then application control), each as its own gated phase with its own validation and rollback.
1. **Expand and decommission** — extend each validated phase from pilot to broad scope, then scope the legacy GPOs away from migrated devices, one authority per setting.

Why sequencing is the whole point (the *why* lives here)

The order is not cosmetic — it is the difference between a clean transition and an outage:

• **A transition is a coexistence problem, not a copy problem.** During the move, the same device can receive settings from *both* Group Policy and Intune. Without a precedence rule and a clean sequence, the two authorities fight: one sets a value, the other overrides it, the device flips back and forth, and you get conflicts and evaluation **loops**.

• **Core OS settings are low blast radius; security controls are high blast radius.** A wrong device restriction is an annoyance. A wrong firewall rule or application-control policy applied while two authorities are still fighting can **lock users out of the network or block the apps they need to work** — and it can do it fleet-wide.

• **You cannot validate a control on an unstable foundation.** If precedence and conflict governance are not proven first, you cannot tell whether a security control "failed" because the control is wrong or because Group Policy is still overriding it. Stabilizing the foundation first makes every later phase *diagnosable*.

The anti-pattern this program exists to prevent

> ***"Internal pressure to migrate security too quickly."*** *Once a migration is underway, there is almost always pressure — from leadership, from security, from the team itself — to "just move the firewall and app control too, while we're in there." This is the single most common way a GPO-to-Intune transition breaks. Security controls migrated **before** the core OS is clean and stable land on top of unresolved precedence conflicts and produce lockouts and loops.* ***This program's job is to hold the line: security controls do not start until the core OS phase has passed its stabilization go/no-go gate.*** *The gate is the governance mechanism that converts "pressure to rush" into "evidence we are ready."*

Coexistence & precedence (MDM wins over GP)

The transition runs both authorities at once across the fleet, so precedence must be explicit:

• **MDM wins over GP.** Windows exposes the `MDMWinsOverGP` control (the `ControlPolicyConflict` policy, configured in Intune as the `MDM Wins Over GP` setting). With it enabled, where Intune and Group Policy set the **same** setting, the **Intune (MDM) value wins**. This is the precedence rule for the entire transition.

• **One authority per setting, end-state.** Coexistence is a *transitional* state, not the goal. As each setting area migrates and validates, the legacy GPO for that area is **scoped away** from migrated devices (security filter / unlink), so the end-state is exactly one authority per setting.

• **Precedence is necessary but not sufficient.** The `MDMWinsOverGP` rule resolves *same-setting* conflicts; it does not resolve settings that exist only in GPO, or two GPOs fighting each other, or a security control applied before its dependencies. Sequencing and rationalization handle those.

Conflict and loop guardrails

The program builds in guardrails so coexistence does not degrade into churn:

• **Migrate by area, scope away the legacy GPO for that area** — never leave both authorities setting the same area indefinitely.

• **Set precedence before you migrate settings** — `MDMWinsOverGP` is deployed as part of the core-OS phase, not after, so the very first migrated settings already have a defined winner.

• **Watch for evaluation loops** — a setting that flips between a GPO value and an Intune value on successive policy cycles is the signature of a loop; the validation script flags it and the remediation script resolves precedence.

• **Never let two phases touch the same setting area at once** — the phase plan assigns each setting area to exactly one phase.

Go/no-go gates per phase

Every phase boundary is a **go/no-go gate**. A phase does not start until the previous phase has passed its gate; security phases do not start until the stabilization gate is green. Each gate is a short, named checklist (see the Implementation and Testing guides) covering: precedence in effect, no unresolved conflicts/loops on the in-scope scope group, parity validated for the phase's settings, and rollback confirmed available. The gate is owned by the program lead, signed off with the stakeholders, and recorded — it is the mechanism that keeps the transition honest under pressure.

The program operating model (roles & KPIs)

Role	Responsibility
Program lead	Owns the phase plan, the go/no-go gates, and the decision to advance or hold.
Endpoint / Intune admin	Builds and assigns the Intune policies; runs the migration mechanics per phase.
Group Policy / AD admin	Rationalizes the GPO backlog; scopes legacy GPOs away as areas migrate.
Security owner	Defines the security-control phases (firewall, app control) — and accepts they come *later*.
Change manager	Schedules windows, owns comms and rollback approval per phase.

Track the transition with these KPIs (the report script feeds them):

• **Phase completion** — phases passed vs. planned.

• **Precedence coverage** — % of in-scope devices with `MDMWinsOverGP` in effect.

• **Open conflicts / loops** — count on the current in-scope scope group (target: zero before the gate).

• **GPO rationalization** — GPOs retired/consolidated vs. baseline backlog.

• **Coexistence cleanup** — setting areas with legacy GPO scoped away (one-authority-per-setting %).

How this complements the rest of the catalog (no duplication)

- It **complements, does not duplicate**, the **Group Policy to Intune Settings Migration** solution,

which provides the per-setting **mechanics** (export, Group Policy analytics, settings catalog, ADMX import, parity). This program decides the **order and governance** of those mechanics.

- It **complements, does not duplicate**, the **firewall** and **application-control** migration

solutions — those are the per-control **products**. This program decides **when** each is safe to start and **how** it is gated within the overall sequence.

Honest limitations

- **This is a program, not a same-day cutover.** A real transition runs weeks to months; the

discipline is the value.

- **It governs the sequence; it does not replace the mechanics or per-control products.** Use it

alongside the settings-migration and per-control solutions.

- **Coexistence is transitional.** Running both authorities indefinitely is the failure mode the

program is designed to end, not sustain.

- **Security must wait.** The model is explicit that core OS comes first; if the organization will

not accept sequencing, this program cannot protect the transition.

The standard deployment model

This solution follows the standard model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — applied as a phased program with named artifacts and a **go/no-go gate at every phase boundary**:

Rationalize → **Core OS (set precedence)** → **Stabilize (gate)** → **Security controls in order (gate each)** → **Expand** → **Decommission GPO scope.**

2. Requirements Guide — Phased GPO-to-Intune Transition Program

Verify **every** requirement before running the program. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites are in `global-requirements-migrations.md`.

> ***Naming convention (mandatory, reused verbatim across the program):*** *every created artifact is* `<Purpose> – <Scope> – <Stage>`. ***Scope*** = `Core OS | Firewall | App Control | All | Identity`; ***Stage*** = `Source` *(GPO)* | `Target` *(Intune)* | `Program`. *Canonical artifacts introduced here are reused in the Implementation Guide verbatim — for example* `Transition Wave – Core OS – Pilot`, `Coexistence Policy – MDM Wins Over GP – All Migrated`, `Phase Gate Checklist – Core OS – Program`.

2.1 Intune licensing & admin access

Description: An Intune-bearing license plan and the admin roles to build and assign device configuration policies (settings catalog, imported ADMX, endpoint security). **Why:** Every migrated setting and every coexistence/precedence policy is built and assigned in Intune; the program cannot configure the target without it. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** active; you hold **Intune Administrator**. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1; grant least-privilege Intune admin roles. (See global requirements G1, G2.)

2.2 Devices enrolled / co-managed in Intune

Description: The in-scope domain-joined devices are Entra-joined or Entra-hybrid-joined and enrolled (or co-managed) in Intune so MDM policy applies. **Why:** Coexistence and precedence (`MDMWinsOverGP`) only take effect on devices Intune actually manages; an un-enrolled device receives Group Policy only. **Verify:** Intune → **Devices** → **All devices**; run `scripts\GPOPROG-Discovery.ps1` (reports Intune coverage vs. the GPO-scoped population). **Implement:** Configure Windows automatic enrollment (Entra → **Mobility (MDM and MAM)**); for co-management, set the relevant workloads toward Intune. (See global requirement G5.)

2.3 A rationalized GPO backlog

Description: The Group Policy estate has been inventoried and **rationalized** — dead, duplicate, and obsolete GPOs identified for retirement — *before* migration begins. **Why:** Migrating an un-rationalized backlog migrates accumulated cruft and inflates conflict surface; you migrate intent, not history. Rationalization is the program's first phase. **Verify:** Run `scripts\GPOPROG-Discovery.ps1` (enumerates GPOs, scope, link/enabled state); confirm a reviewed retire/keep/consolidate decision exists per GPO. **Implement:** Use the **GroupPolicy** module export (`Get-GPO` / `Get-GPOReport`) and GPMC to review every GPO; record a retire/keep/consolidate decision per GPO as the rationalization backlog.

2.4 Group Policy analytics access (for the mechanics phases)

Description: Access to Intune **Group Policy analytics** to import GPO backups and read per-GPO migration-readiness and per-setting mapping. **Why:** The program sequences the *mechanics* solution; readiness % drives which GPOs are migrate-first (high readiness) vs. plan-handling (many unsupported settings). **Verify:** Intune → **Reports** → **Group Policy analytics** loads and accepts a GPO backup import. **Implement:** Ensure the role includes report access; produce GPO backups with `scripts\GPOPROG-Discovery.ps1` or GPMC **Back Up All**.

2.5 Coexistence precedence decision (MDM wins over GP)

Description: A documented decision to run the transition with **MDM wins over GP** as the precedence rule, deployed as a coexistence policy in the core-OS phase. **Why:** Coexistence runs both authorities at once; without an explicit precedence rule, same-setting conflicts produce evaluation loops. `MDMWinsOverGP` makes the Intune value the winner. **Verify:** Run `scripts\GPOPROG-Validation.ps1` on a pilot device (reports `MDMWinsOverGP` state); confirm `Coexistence Policy – MDM Wins Over GP – All Migrated` is planned/assigned. **Implement:** Build the settings-catalog policy `Coexistence Policy – MDM Wins Over GP – All Migrated` (the `MDM Wins Over GP` / `ControlPolicyConflict` setting) and assign it to the core-OS pilot scope first. (See Implementation Step 3.)

2.6 Phase scope / wave groups

Description: Microsoft Entra security groups for each phase's pilot and broad scope, named to the convention (for example `Transition Wave – Core OS – Pilot`). **Why:** Each phase is piloted on a small scope, gated, then expanded; phase-scoped groups are how policy is targeted phase-by-phase without bleed between phases. **Verify:** Entra → **Groups**; or run `scripts\GPOPROG-Set-PhaseScopeTargeting.ps1` (creates/reuses the phase scope groups). **Implement:** Create the groups with `scripts\GPOPROG-Set-PhaseScopeTargeting.ps1`. (See global requirement G6.)

2.7 Change control, coexistence window & per-phase rollback

Description: A change window per phase, a defined coexistence approach (both authorities managed during transition), and a documented rollback per phase (re-scope the legacy GPO back / revert the Intune assignment). **Why:** The transition touches live, domain-joined production devices; each phase — especially the security phases — must be reversible and coordinated. **Verify:** A rollback procedure and coexistence plan are written and approved **per phase** before that phase starts. **Implement:** Define rollback per phase (legacy GPO stays present-but-unlinked until the phase is confirmed), a comms plan, and the order of operations. (See global requirement G7.)

2.8 Stakeholder alignment on sequencing (security comes later)

Description: Documented agreement from leadership and the security owner that **core OS migrates first and security controls come later**, gated. **Why:** This is the governance backbone. Without it, internal pressure to rush security defeats the program; the alignment is what the go/no-go gate enforces. **Verify:** A signed sequencing decision exists naming the phase order and the gate owner. **Implement:** Hold the sequencing alignment session; record the phase order, gate criteria, and the program lead as gate owner before Phase 1.

Checklist

- Intune licensing + Intune Administrator role
- In-scope devices enrolled / co-managed — `GPOPROG-Discovery.ps1`
- GPO backlog inventoried and rationalized (retire/keep/consolidate per GPO)
- Group Policy analytics access (for the mechanics phases)
- MDM-wins-over-GP precedence decided — `Coexistence Policy – MDM Wins Over GP – All Migrated`
- Phase scope/wave groups created — `GPOPROG-Set-PhaseScopeTargeting.ps1`
- Per-phase change control, coexistence window, and rollback approved
- Stakeholder sign-off: core OS first, security later, gated

3. Implementation Guide — Phased GPO-to-Intune Transition Program

This is the **program runbook**. It follows the standard model — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — applied as a sequenced, gated program. GPO inventory/export is done in the **Group Policy Management Console (GPMC)** on a domain-joined admin workstation; everything else is in the **Microsoft Intune admin center** (`https://intune.microsoft.com`) and **Microsoft Entra** (`https://entra.microsoft.com`).

> ***Scope boundary (read first):*** *this program governs the* ***sequence and the gates****. The per-setting migration* ***mechanics*** *(Group Policy analytics, settings catalog, ADMX import, per-setting parity) live in the* ***Group Policy to Intune Settings Migration*** *solution; the per-control build-outs live in the* ***firewall*** *and* ***application-control*** *migration solutions. This runbook says *what order* they run in and *when each is safe to start* — it does not re-document those mechanics.*

> ***Naming convention (mandatory, reused verbatim):*** *every created artifact is* `<Purpose> – <Scope> – <Stage>`. ***Scope*** = `Core OS | Firewall | App Control | All | Identity`; ***Stage*** = `Source` *(GPO)* | `Target` *(Intune)* | `Program`. *Canonical artifacts:* `Transition Wave – <Scope> – Pilot`, `Transition Wave – <Scope> – Broad`, `Coexistence Policy – MDM Wins Over GP – All Migrated`, `Phase Gate Checklist – <Scope> – Program`, `Rationalization Backlog – All – Source`.

The phase sequence (do not reorder)

#	Phase	Goal	Gate to advance
0	**Rationalize**	Retire/consolidate the GPO backlog; baseline both sides.	Rationalization backlog signed off.
1	**Core OS — set precedence + migrate foundation**	Deploy `MDMWinsOverGP`; migrate low-blast-radius OS settings on a pilot.	Precedence in effect; core-OS parity validated on pilot.
2	**Stabilize**	Prove no conflicts/loops; expand core OS to broad.	**Stabilization gate** — zero open conflicts/loops on broad core-OS scope.
3	**Security control A — Firewall**	Only after Phase 2 passes: sequence the firewall migration, gated.	Firewall parity validated; no lockouts on pilot, then broad.
4	**Security control B — App control**	Only after Phase 3 passes: sequence application control, gated.	App-control parity validated; no block-outs on pilot, then broad.
5	**Expand & decommission**	Scope legacy GPOs away from migrated devices; one authority per setting.	All phases passed; legacy GPOs unlinked; evidence archived.

> ***The line the program holds:*** *Phases 3 and 4* ***do not start*** *until the Phase 2 stabilization gate is green — regardless of internal pressure to "move security too." The gate, not the pressure, decides.*

Phase 0 — Rationalize (Assess)

Goal: migrate intent, not accumulated cruft; baseline both authorities.

1. **Inventory the GPO estate.** Run the read-only `scripts\GPOPROG-Discovery.ps1` on a domain-joined

admin workstation — it enumerates every GPO (scope, link state, enabled state) via the **GroupPolicy** module, reports the `MDMWinsOverGP` precedence state, and reports Intune coverage of the in-scope population.

1. **Rationalize.** For each GPO record a **retire / keep / consolidate** decision into

`Rationalization Backlog – All – Source`. Retire dead/duplicate/obsolete policy *before* migrating.

1. **Group settings into phases.** Assign every *kept* setting area to exactly one phase: foundational

OS settings → Phase 1 (Core OS); firewall → Phase 3; application control → Phase 4. **No setting area belongs to two phases.**

1. **Export for the mechanics.** Produce GPO backups/XML (the script's export, or GPMC **Back Up

All**) as the input for Group Policy analytics in the later mechanics phases — and as your retained rollback source.

Exit gate (Phase 0): a signed `Rationalization Backlog – All – Source`, a per-phase setting-area assignment, and the Intune-coverage baseline.

Phase 1 — Core OS: set precedence, then migrate the foundation (Configure + Pilot)

Goal: establish the precedence rule, then migrate the lowest-blast-radius settings on a pilot.

Step 1 — Build the phase scope groups

Run `scripts\GPOPROG-Set-PhaseScopeTargeting.ps1` to create (idempotently) the Entra security groups for this phase: `Transition Wave – Core OS – Pilot` (a few representative devices) and `Transition Wave – Core OS – Broad`. Reuse these names verbatim in every assignment.

Step 2 — Confirm enrollment & coverage

Run `scripts\GPOPROG-Discovery.ps1` and confirm the pilot devices are Intune-managed (precedence and migrated settings only apply to managed devices).

Step 3 — Deploy the precedence policy FIRST (`MDMWinsOverGP`)

Set the winner *before* migrating settings, so the very first migrated settings already have a defined authority:

1. Intune → **Devices → Configuration → Create → New policy → Windows 10 and later → Settings

catalog**.

1. Search **MDM Wins Over GP** (the `ControlPolicyConflict` setting) and set it so **MDM wins**.
2. Name it `Coexistence Policy – MDM Wins Over GP – All Migrated`.
3. **Assignments** → **Add groups** → `Transition Wave – Core OS – Pilot` (extended to broad in Phase 2).

Create.

Step 4 — Migrate the core-OS settings (via the mechanics solution)

Using the **Group Policy to Intune Settings Migration** mechanics (Group Policy analytics → settings catalog / imported ADMX), migrate **only the Phase-1 core-OS setting areas**. Name the resulting configuration `Settings Catalog – Core OS – Target` and assign it to `Transition Wave – Core OS – Pilot`. (This program does not re-document the mechanics — it scopes them to the core-OS areas only.)

Step 5 — Scope the legacy core-OS GPO away from the pilot

As each core-OS area applies in Intune, **security-filter / unlink** the corresponding legacy GPO for the pilot devices so there is one authority per migrated area (leave the GPO present-but-unlinked for rollback).

Step 6 — Validate the core-OS pilot

1. On a pilot device, sync policy (**Settings** → **Accounts** → **Access work or school** → **Info** → **Sync**).
2. Run `scripts\GPOPROG-Validation.ps1 -Phase "Core OS"` → confirms `MDMWinsOverGP` is in effect,

reports applied MDM settings, and flags any same-setting conflicts or flip/loop signatures for the phase.

1. Run `scripts\GPOPROG-Report.ps1` → phase status + conflict report.
2. Complete the **Testing Guide** core-OS gate. **Do not advance until the Phase-1 gate passes.**

Exit gate (Phase 1): `MDMWinsOverGP` ENABLED on pilot; core-OS parity validated; zero unresolved conflicts/loops on the pilot scope.

Phase 2 — Stabilize (Validate + Expand core OS) — THE security go/no-go gate

Goal: prove the foundation is stable on the broad fleet before any security control is touched.

1. **Expand precedence + core OS to broad.** Add `Transition Wave – Core OS – Broad` to `Coexistence Policy – MDM Wins Over GP – All Migrated` and to `Settings Catalog – Core OS – Target`. Move devices into the broad group in waves.

1. **Scope legacy core-OS GPOs away** from the broad migrated devices (still present-but-unlinked for rollback).

1. **Stabilization soak.** Let the broad scope run for a defined soak period (at least one full policy refresh cycle, ideally several) so conflicts/loops have time to surface.

1. **Validate broad.** Run `scripts\GPOPROG-Validation.ps1 -Phase "Core OS"` and

`scripts\GPOPROG-Report.ps1` across the broad scope. The conflict count must be **zero** and no loop signatures present.

1. **Hold the stabilization gate.** This is the program's central go/no-go. It passes **only** when:

precedence is in effect fleet-wide, core-OS parity is validated, and there are **zero** open conflicts/loops on the broad core-OS scope. The program lead signs off `Phase Gate Checklist – Core OS – Program` with the stakeholders.

> ***Anti-pattern guard:*** *if there is pressure to start firewall or app-control migration before this gate is green, the answer is* ***no****. Security controls layered on an unstable foundation produce lockouts and loops. The gate is the evidence that converts "we feel ready" into "we are ready."*

Exit gate (Phase 2 — the security go/no-go): zero conflicts/loops on broad core OS; precedence fleet-wide; `Phase Gate Checklist – Core OS – Program` signed. Only now may Phase 3 begin.

Phase 3 — Security control A: Firewall (gated)

Goal: sequence the firewall migration *after* the foundation is stable, behind its own gate.

1. **Build the phase scope groups:** `scripts\GPOPROG-Set-PhaseScopeTargeting.ps1 -Phase "Firewall"`

→ `Transition Wave – Firewall – Pilot` and `Transition Wave – Firewall – Broad`.

1. **Run the firewall migration** using the dedicated **firewall migration** solution (this program

does not re-document its mechanics). Name the resulting Intune policy `Endpoint Security – Firewall – Target` and assign to `Transition Wave – Firewall – Pilot`.

1. **Scope the legacy firewall GPO away** from the firewall pilot devices (present-but-unlinked for

rollback).

1. **Validate the firewall pilot** — run `scripts\GPOPROG-Validation.ps1 -Phase "Firewall"` and

`scripts\GPOPROG-Report.ps1`; confirm no precedence conflicts and — critically — **no network lockouts** (users can still reach what they need). Complete the Testing Guide firewall gate.

1. **Expand to broad** only after the pilot passes; re-validate the broad scope.

Exit gate (Phase 3): firewall parity validated; zero conflicts/loops; no lockouts on pilot then broad; `Phase Gate Checklist – Firewall – Program` signed.

Phase 4 — Security control B: Application control (gated)

Goal: sequence application control **last** among the security controls, behind its own gate.

1. **Build the phase scope groups:** `scripts\GPOPROG-Set-PhaseScopeTargeting.ps1 -Phase "App Control"` → `Transition Wave – App Control – Pilot` and `Transition Wave – App Control – Broad`.

1. **Run the application-control migration** using the dedicated **application-control** solution.

Name the resulting policy `Endpoint Security – App Control – Target` and assign to `Transition Wave – App Control – Pilot`. Application control has the highest block-out risk — pilot in **audit/learn mode** first where the control supports it.

1. **Scope the legacy app-control GPO away** from the app-control pilot devices (present-but-unlinked).
2. **Validate the app-control pilot** — run `scripts\GPOPROG-Validation.ps1 -Phase "App Control"` and

`scripts\GPOPROG-Report.ps1`; confirm no precedence conflicts and **no block-outs of required apps**. Complete the Testing Guide app-control gate.

1. **Expand to broad** only after the pilot passes (and after any audit-mode learning is reviewed);

re-validate the broad scope.

Exit gate (Phase 4): app-control parity validated; zero conflicts/loops; no required-app block-outs on pilot then broad; `Phase Gate Checklist – App Control – Program` signed.

Phase 5 — Expand & decommission GPO scope

Goal: end coexistence — one authority per setting — under change control.

1. **Confirm all phases passed** (`scripts\GPOPROG-Report.ps1` shows every phase green and zero open

conflicts/loops).

1. **Scope legacy GPOs away permanently** from the fully migrated fleet — but keep them

present-but-unlinked for a defined grace period so per-phase rollback stays available.

1. **Confirm one authority per setting** — run `scripts\GPOPROG-Validation.ps1` across the fleet; no

setting area should still be driven by both authorities.

1. **Retire the legacy GPOs** after the grace period elapses, with the AD/GPO owner under change

control.

1. **Archive evidence** — the rationalization backlog, every `Phase Gate Checklist – <Scope> –

Program`, and the final report — and close the program.

Exit gate (Phase 5): legacy GPOs retired (or unlinked through grace period); one authority per setting confirmed; evidence archived; program closed.

Order of operations

`Rationalize → Core OS (set MDMWinsOverGP first → migrate foundation → validate pilot) → Stabilize (expand broad → soak → STABILIZATION GATE) → Firewall (gated) → App control (gated) → Expand & decommission GPO scope`

5. Testing Guide — Phased GPO-to-Intune Transition Program

Run the gate for each phase **before advancing to the next**. Every phase boundary is a go/no-go gate; the program does not skip ahead. Use `scripts\GPOPROG-Validation.ps1 -Phase "<Phase>"` on a device in that phase's scope and `scripts\GPOPROG-Report.ps1` for the phase-status + conflict report.

Test 1 — Precedence in effect (MDM wins over GP)

Do: On a Phase-1 pilot device run `GPOPROG-Validation.ps1 -Phase "Core OS"`. **Expected:** `MDMWinsOverGP` reports **ENABLED**; where Intune and GPO set the same setting, the Intune value is the applied value.

Test 2 — Core-OS parity on pilot

Do: Run `GPOPROG-Validation.ps1 -Phase "Core OS"` (optionally with the GPO XML export for the migrated areas). **Expected:** Each migrated core-OS setting is applied by MDM; no migrated area is still driven by the legacy GPO on the pilot device.

Test 3 — No conflicts or evaluation loops

Do: Run `GPOPROG-Validation.ps1` across the phase scope, then re-run after a policy cycle. **Expected:** Zero same-setting conflicts; **no flip/loop signature** (no setting alternating between a GPO value and an Intune value across cycles).

Test 4 — Stabilization gate (the security go/no-go)

Do: After core OS is expanded to broad and soaked, run `GPOPROG-Validation.ps1 -Phase "Core OS"` and `GPOPROG-Report.ps1` across `Transition Wave – Core OS – Broad`. **Expected:** Precedence in effect fleet-wide; core-OS parity validated; **zero** open conflicts/loops on the broad scope. Only when this passes may any security phase begin.

Test 5 — Firewall phase: parity + no lockouts

Do: On a `Transition Wave – Firewall – Pilot` device run `GPOPROG-Validation.ps1 -Phase "Firewall"`; confirm required network access still works. **Expected:** Firewall policy applied by Intune; no precedence conflict; **no network lockout** of required services. Repeat for broad before advancing.

Test 6 — App-control phase: parity + no required-app block-outs

Do: On a `Transition Wave – App Control – Pilot` device run `GPOPROG-Validation.ps1 -Phase "App Control"`; confirm required apps still launch (review audit-mode events first where used). **Expected:** App-control policy applied by Intune; no precedence conflict; **no block-out** of required apps. Repeat for broad before advancing.

Test 7 — One authority per setting (decommission readiness)

Do: Run `GPOPROG-Validation.ps1` across the fleet and `GPOPROG-Report.ps1`. **Expected:** No migrated setting area is still driven by both authorities; legacy GPOs for migrated areas are unlinked/scoped away.

Test 8 — Phase report / KPIs

Do: Run `GPOPROG-Report.ps1`. **Expected:** A per-phase status table (phase, scope group, precedence coverage, open conflicts/loops, gate state) plus the conflict report — feeding the program KPIs.

Pass/fail gates

Gate	Requirement
Phase 1 (Core OS pilot)	Tests 1–3 pass on the core-OS pilot
Phase 2 (Stabilization — security go/no-go)	Test 4 passes: zero conflicts/loops on broad core OS, precedence fleet-wide. **Security phases blocked until green.**
Phase 3 (Firewall)	Test 5 passes on pilot then broad (parity, no lockouts)
Phase 4 (App control)	Test 6 passes on pilot then broad (parity, no required-app block-outs)
Phase 5 (Decommission)	Tests 7–8 pass: one authority per setting, all phases green

6. Troubleshooting Guide — Phased GPO-to-Intune Transition Program

Use `scripts\GPOPROG-Discovery.ps1` for transition readiness, `scripts\GPOPROG-Validation.ps1 -Phase "<Phase>"` for per-phase coexistence/conflict state, `scripts\GPOPROG-Report.ps1` for phase status and the conflict report, and `scripts\GPOPROG-Remediation.ps1` for safe precedence fixes.

A. Settings flip between GPO and Intune (the loop signature)

Symptom	Cause	Remediation
A setting alternates between a GPO value and an Intune value across policy cycles	No precedence set, or the legacy GPO for a migrated area is still linked	Run `GPOPROG-Remediation.ps1` (sets `MDMWinsOverGP`, requests sync); then **scope the legacy GPO away** for that area — the durable fix is one authority per setting

Symptom	Cause	Remediation
`GPOPROG-Validation.ps1` reports `MDMWinsOverGP : not enabled`	`Coexistence Policy – MDM Wins Over GP – All Migrated` not assigned to the device	Assign the coexistence policy to the device's phase scope group; sync

B. Same-setting conflicts on a migrated area

• Confirm precedence is in effect (Test 1), then confirm the **legacy GPO for that area is unlinked**

from migrated devices. A conflict that persists with `MDMWinsOverGP` enabled almost always means the Intune value and the GPO value differ *and* both still apply — remove the GPO authority for that area.

C. Pressure to start security before stabilization passes

• This is a governance issue, not a technical one. **Hold the Phase-2 stabilization gate.** If

`GPOPROG-Report.ps1` shows any open conflict/loop on the broad core-OS scope, security phases stay blocked. Surface the open count and the gate checklist to stakeholders; the gate, not the pressure, decides.

D. Firewall phase: network lockout after migration

Symptom	Cause	Remediation
Users lose required network access after the firewall policy applies	Firewall control migrated before core OS was stable, or a rule gap vs. the legacy GPO	**Roll the firewall phase back** (re-link the legacy firewall GPO, which is still present-but-unlinked; revert the Intune assignment); re-confirm Phase-2 stabilization; re-run the firewall pilot against parity

E. App-control phase: required apps blocked

Symptom	Cause	Remediation
Required applications won't launch after app-control policy applies	App control enforced before audit/learn review, or a rule gap	Switch the control to **audit/learn** mode, review the block events, fill the rule gap, re-validate the pilot, then re-enforce; roll back via the present-but-unlinked legacy GPO if needed

F. A phase won't validate / gate stays red

• Confirm the phase scope group membership is correct and devices are Intune-managed

(`GPOPROG-Discovery.ps1`). A device receiving GPO only (not enrolled) cannot honor precedence — enroll it or remove it from the phase scope.

G. Diagnostics quick reference

```
# Precedence (MDM wins over GP) on the device
Get-ItemProperty 'HKLM:\SOFTWARE\Microsoft\PolicyManager\current\device\ControlPolicyConflict' -EA Silentl
# What Intune is enforcing (applied MDM PolicyManager values)
Get-ChildItem 'HKLM:\SOFTWARE\Microsoft\PolicyManager\current\device' -EA SilentlyContinue
# Resultant Group Policy (what GPO still applies) — review against migrated areas
gpresult /h .\rsop.html
# Trigger an MDM/Intune sync so policy re-evaluates
Start-Process "$env:WINDIR\System32\deviceenroller.exe" -ArgumentList "/c /AutoEnrollMDM"
```

H. When to escalate

If precedence is enabled, the legacy GPO for the area is unlinked, the device is enrolled and synced, and a conflict/loop still persists after a policy cycle, capture `gpresult /h` output plus `GPOPROG-Validation.ps1` results and open a Microsoft support case before advancing the affected phase.

7. Best Practices — Phased GPO-to-Intune Transition Program

• **Core OS first, security later — always.** The foundation must be migrated and stable before any security control (firewall, application control) is touched. This single sequencing rule prevents most transition failures.

• **Hold the stabilization gate against pressure.** The most common way a transition breaks is internal pressure to migrate security too quickly. Treat the Phase-2 go/no-go gate as non-negotiable: security phases do not start until the broad core-OS scope shows zero conflicts/loops.

• **Set precedence before you migrate settings.** Deploy `Coexistence Policy – MDM Wins Over GP – All Migrated` *first*, so the very first migrated settings already have a defined winner — never migrate settings into an undefined-precedence state.

• **Rationalize before you migrate.** Retire dead, duplicate, and obsolete GPOs first; migrate intent, not accumulated cruft. Less surface means fewer conflicts.

• **One setting area, one phase.** Assign every setting area to exactly one phase; never let two phases touch the same area, and never leave both authorities driving the same area indefinitely.

• **Scope the legacy GPO away as each area migrates.** Coexistence is transitional. The end-state is one authority per setting — security-filter / unlink the legacy GPO for each migrated area (keep it present-but-unlinked for rollback).

• **Watch for loops, not just conflicts.** A setting that flips between a GPO value and an Intune value across policy cycles is the loop signature; the validation script flags it — resolve precedence before advancing.

• **Pilot, soak, then expand — every phase.** Each phase gets its own pilot, a stabilization soak

across at least one full policy cycle, and a re-validate before broad expansion.

• **Pilot application control in audit/learn mode.** App control has the highest block-out risk; learn first, review events, then enforce.

• **Gate, sign, and record.** Every phase boundary is a named `Phase Gate Checklist – <Scope> – Program`, signed by the program lead with stakeholders — the audit trail and the pressure valve.

• **Name everything to the convention.** `<Purpose> – <Scope> – <Stage>`, reused verbatim, so status, assignments, and rollback are unambiguous.

• **Keep rollback available through a grace period.** Leave legacy GPOs present-but-unlinked until each phase is confirmed; decommission only after the grace period.

• **Track the KPIs.** Phase completion, precedence coverage, open conflicts/loops, GPO rationalization, and coexistence cleanup — a downward conflict count and rising one-authority-per-setting % mean the transition is healthy.

8. Deliverables Summary — Phased GPO-to-Intune Transition Program

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The phased model, coexistence/precedence (MDM wins over GP), conflict/loop governance, go/no-go gates, and the program operating model — and the *why* of sequencing
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Migrations)	Category-wide prerequisites, published once
3. Implementation Guide	The ordered program runbook: rationalize → core OS (set precedence) → stabilize (gate) → security controls in order (gate each) → expand & decommission
5. Testing Guide	Per-phase validation with pass/fail go/no-go gates
6. Troubleshooting Guide	Field issues (conflicts, loops, lockouts) and remediation
7. Best Practices	Field-proven recommendations for the operating model

PowerShell scripts

Script	What it does
`GPOPROG-Discovery.ps1`	Transition readiness: GPO scope/inventory, `MDMWinsOverGP` precedence state, and Intune coverage of the in-scope population (read-only)
`GPOPROG-Validation.ps1`	Per-phase coexistence/conflict validation: precedence in effect, applied MDM settings, same-setting conflicts and flip/loop signatures (read-only)
`GPOPROG-Report.ps1`	Phase-status + conflict report and program KPIs (read-only; Microsoft Graph optional)
`GPOPROG-Remediation.ps1`	Safe remediation of precedence conflicts (sets `MDMWinsOverGP`, requests sync); `-WhatIf` by default, changes only with `-Apply`
`GPOPROG-Set-PhaseScopeTargeting.ps1`	Builds the phase/wave scope groups in Entra to the naming convention (Microsoft Graph)

What the buyer receives

- This program guide as a branded **Word** document and **PDF**.
- All five **PowerShell scripts** in a single ZIP.
- A repeatable, gated program following the model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

How this fits the catalog (complements, does not duplicate)

- The **Group Policy to Intune Settings Migration** solution provides the per-setting **mechanics**;

this program sequences and governs them.

- The **firewall** and **application-control** migration solutions are the per-control **products**;

this program decides when each is safe to start and how it is gated.

Outcome

A sequenced, governed GPO-to-Intune transition — foundation first, security next, gated throughout — that holds together under pressure, stays reversible per phase, and ends with one authority per setting.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GPOPROG-Discovery.ps1

Purpose: Read-only readiness baseline for the transition program. On a domain-joined admin workstation it uses the GroupPolicy module to enumerate every GPO (scope, link state, enabled state) as the rationalization backlog input; it reports the local device's MDMWinsOverGP precedence state; and (with -UseGraph) it reports Intune coverage of the in-scope device population via Microsoft Graph. Nothing is changed. HOW TO RUN On a domain-joined admin workstation (RSAT Group Policy Management Tools installed): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Discovery.ps1 -OutputPath .\GPOPROG-Out Add Intune coverage (Microsoft.Graph module + DeviceManagementManagedDevices.Read.All): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Discovery.ps1 -UseGraph EXPECTED RESULTS A GPO inventory table + CSV (DisplayName, Id, GpoStatus, link/enabled state) for rationalization, the local MDMWinsOverGP precedence state, and — with -UseGraph — a count of Intune-managed Windows devices (coverage signal). Read-only. .NOTES Migrations. Read-only. Requires the GroupPolicy module for the GPO side (Add-WindowsCapability -Online -Name Rsat.GroupPolicy.Management.Tools~~~~0.0.1.0). > [CmdletBinding()] param([string] $OutputPath = ".\GPOPROG-Out", [switch] $UseGraph)

How to run: On a domain-joined admin workstation (RSAT Group Policy Management Tools installed): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Discovery.ps1 -OutputPath .\GPOPROG-Out Add Intune coverage (Microsoft.Graph module + DeviceManagementManagedDevices.Read.All): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Discovery.ps1 -UseGraph EXPECTED RESULTS A GPO inventory table + CSV (DisplayName, Id, GpoStatus, link/enabled state) for rationalization, the local MDMWinsOverGP precedence state, and — with -UseGraph — a count of Intune-managed Windows devices (coverage signal). Read-only. .NOTES Migrations. Read-only. Requires the GroupPolicy module for the GPO side (Add-WindowsCapability -Online -Name Rsat.GroupPolicy.Management.Tools~~~~0.0.1.0). > [CmdletBinding()] param([string] $OutputPath = ".\GPOPROG-Out", [switch] $UseGraph)

GPOPROG-Remediation.ps1

Purpose: Resolves the most common transition conflict: Group Policy still winning over migrated Intune settings (the precedence fight behind same-setting conflicts and evaluation loops). It sets the local MDMWinsOverGP (ControlPolicyConflict) value so MDM/Intune wins, then requests an MDM sync so policy re-evaluates. Runs in -WhatIf mode by DEFAULT and changes nothing until you pass -Apply. The durable fix is the Intune policy 'Coexistence Policy - MDM Wins Over GP - All Migrated' plus scoping the legacy GPO away for the affected area; this script unblocks a device for per-phase testing. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Remediation.ps1 Apply on a phase-scope pilot device (elevated): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Remediation.ps1 -Apply EXPECTED RESULTS In preview: prints what WOULD change (MDMWinsOverGP -> 1) and exits. With -Apply: sets MDMWinsOverGP=1, requests an MDM sync, and advises re-running GPOPROG-Validation.ps1 and scoping the legacy GPO away for the conflicting area. .NOTES Migrations. -WhatIf by default; -Apply makes changes. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch] $Apply)

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Remediation.ps1 Apply on a phase-scope pilot device (elevated): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Remediation.ps1 -Apply EXPECTED RESULTS In preview: prints what WOULD change (MDMWinsOverGP -> 1) and exits. With -Apply: sets MDMWinsOverGP=1, requests an MDM sync, and advises re-running GPOPROG-Validation.ps1 and scoping the legacy GPO away for the conflicting area. .NOTES Migrations. -WhatIf by default; -Apply makes changes. > [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([switch] $Apply)

GPOPROG-Report.ps1

Purpose: Aggregates the program's status. It reads the per-phase validation snapshots produced by GPOPROG-Validation.ps1 (GPOPROG-Validation-*.json) to report, per phase, the precedence state and any loop/conflict findings; and (with -UseGraph) lists the phase scope groups and the Intune transition policies named to the convention (Coexistence Policy / Settings Catalog / Endpoint Security) with their assignments. Produces the gate-readiness view and program KPIs. Read-only. HOW TO RUN From the folder holding the validation snapshots: powershell -ExecutionPolicy Bypass -File .\GPOPROG-Report.ps1 -SnapshotPath . Add the Intune-side view (Microsoft.Graph + DeviceManagementConfiguration.Read.All, Group.Read.All): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Report.ps1 -UseGraph EXPECTED RESULTS A per-phase table (phase, precedence state, loop/conflict count, gate state) and KPIs (precedence coverage, open conflicts/loops). With -UseGraph: the phase scope groups and the named transition policies + assignments. Read-only. .NOTES Migrations. Read-only. > [CmdletBinding()] param([string] $SnapshotPath = ".", [switch] $UseGraph)

How to run: From the folder holding the validation snapshots: powershell -ExecutionPolicy Bypass -File .\GPOPROG-Report.ps1 -SnapshotPath . Add the Intune-side view (Microsoft.Graph + DeviceManagementConfiguration.Read.All, Group.Read.All): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Report.ps1 -UseGraph EXPECTED RESULTS A per-phase table (phase, precedence state, loop/conflict count, gate state) and KPIs (precedence coverage, open conflicts/loops). With -UseGraph: the phase scope groups and the named transition policies + assignments. Read-only. .NOTES Migrations. Read-only. > [CmdletBinding()] param([string] $SnapshotPath = ".", [switch] $UseGraph)

GPOPROG-Set-PhaseScopeTargeting.ps1

Purpose: Creates (idempotently) the two Entra security groups used to target one transition phase — the pilot and broad scope — named to the solution convention, and optionally seeds the pilot group with device members: 'Transition Wave - <Phase> - Pilot' 'Transition Wave - <Phase> - Broad' Existing groups with the same name are reused, not duplicated. Use the resulting names verbatim in the Intune assignments from the Implementation Guide. HOW TO RUN Core OS phase (default): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Set-PhaseScopeTargeting.ps1 -Phase "Core OS" A security phase, seeding pilot devices: powershell -ExecutionPolicy Bypass -File .\GPOPROG-Set-PhaseScopeTargeting.ps1 -Phase "Firewall" -PilotDeviceIds 'objId1','objId2' Requires the Microsoft.Graph module and Group.ReadWrite.All (admin consent). EXPECTED RESULTS Both phase scope groups exist (created or found) with their object IDs printed; any supplied pilot device object IDs are added to the pilot group. .NOTES Migrations. Creates/updates Entra groups only. > [CmdletBinding(SupportsShouldProcess = $true)] param([ValidateSet('Core OS','Firewall','App Control')] [string] $Phase = 'Core OS', [string[]] $PilotDeviceIds = @())

How to run: Core OS phase (default): powershell -ExecutionPolicy Bypass -File .\GPOPROG-Set-PhaseScopeTargeting.ps1 -Phase "Core OS" A security phase, seeding pilot devices:

powershell -ExecutionPolicy Bypass -File .\GPOPROG-Set-PhaseScopeTargeting.ps1 -Phase "Firewall" -PilotDeviceIds 'objId1','objId2' Requires the Microsoft.Graph module and Group.ReadWrite.All (admin consent). EXPECTED RESULTS Both phase scope groups exist (created or found) with their object IDs printed; any supplied pilot device object IDs are added to the pilot group. .NOTES Migrations. Creates/updates Entra groups only. > [CmdletBinding(SupportsShouldProcess = $true)] param([ValidateSet('Core OS','Firewall','App Control')] [string] $Phase = 'Core OS', [string[]] $PilotDeviceIds = @())

GPOPROG-Validation.ps1

Purpose: Confirms a transition phase is safe to advance. On a device in the phase's scope it: reads the MDMWinsOverGP (ControlPolicyConflict) precedence state; lists the applied MDM PolicyManager values Intune is enforcing; and (when -PreviousSnapshot is supplied) compares against a prior run to detect FLIP/LOOP signatures (a setting whose value changed between cycles = the loop signature this program guards against). Writes a snapshot for the next run. Read-only against the device. HOW TO RUN On a phase-scope device (elevated). First run captures a baseline snapshot: powershell -ExecutionPolicy Bypass -File .\GPOPROG-Validation.ps1 -Phase "Core OS" After a policy cycle, compare to detect loops: powershell -ExecutionPolicy Bypass -File .\GPOPROG-Validation.ps1 -Phase "Core OS" -PreviousSnapshot .\GPOPROG-Validation-CoreOS.json EXPECTED RESULTS MDMWinsOverGP state (enabled = pass), the count of applied MDM settings, and — with a previous snapshot — a list of any settings that CHANGED value between cycles (loop signature). Zero changed settings + precedence enabled = the phase coexistence check passes. .NOTES Migrations. Read-only. > [CmdletBinding()] param([string] $Phase = "Core OS", [string] $PreviousSnapshot, [string] $OutputPath = ".")

How to run: On a phase-scope device (elevated). First run captures a baseline snapshot: powershell -ExecutionPolicy Bypass -File .\GPOPROG-Validation.ps1 -Phase "Core OS" After a policy cycle, compare to detect loops: powershell -ExecutionPolicy Bypass -File .\GPOPROG-Validation.ps1 -Phase "Core OS" -PreviousSnapshot .\GPOPROG-Validation-CoreOS.json EXPECTED RESULTS MDMWinsOverGP state (enabled = pass), the count of applied MDM settings, and — with a previous snapshot — a list of any settings that CHANGED value between cycles (loop signature). Zero changed settings + precedence enabled = the phase coexistence check passes. .NOTES Migrations. Read-only. > [CmdletBinding()] param([string] $Phase = "Core OS", [string] $PreviousSnapshot, [string] $OutputPath = ".")